

Project Report: CSS Wildcard Selectors

OS: Arch Linux x86_64 • Engine: Mozilla Firefox & GeckoDriver • Date: September 2026

1 Executive Summary

Modern front-end component architectures (e.g., React, Angular, and Vue) frequently generate dynamic, non-deterministic Document Object Model (DOM) attributes. Elements often receive generated identifiers containing auto-incrementing integers, build-specific hashing, or shared functional prefixes (such as `checkBoxOption1` or `ember-1049`). Rigid locators that bind to entire literal strings fail rapidly across builds, rendering automated test suites fragile and costly to maintain.

This engineering report investigates and validates the deployment of **CSS3 Wildcard Attribute Selectors** to match dynamic DOM nodes resiliently. Executed in a native **Arch Linux** environment using **Selenium WebDriver 4.x** and **Mozilla Firefox** managed via **GeckoDriver**, the test harness demonstrates prefix matching (`^=`), substring containment (`*=`), and suffix matching (`$=`) against the Rahul Shetty Academy Automation Practice testbed. All wildcard queries resolved their target elements with zero false positives, demonstrated interactive state toggles across collections, and maintained clean lifecycle boundaries.

2 System Architecture & Environment Topology

To ensure deterministic execution and avoid wrapper overhead, the automation script was executed directly on a rolling-release Arch Linux system. Driver resolution prioritized system-native binaries before falling back to Selenium Manager.

Component Layer	Deployment Specification	Operational Role
Host Platform	Arch Linux (x86_64 kernel rolling)	Native execution environment and process supervisor
Rendering Engine	Mozilla Firefox (pacman -S firefox)	Gecko layout and CSS selector computation engine
Wire Protocol Bridge	GeckoDriver (/usr/bin/geckodriver)	Native W3C WebDriver communication daemon
Runtime Engine	Python 3.x (.venv virtualenv)	Test workflow orchestration and data parsing
Automation API	Selenium WebDriver 4.x	CSS selector compilation and DOM dispatch
Target SUT	Automation Practice Portal	Canonical benchmark for dynamic UI components

Table 1: Test runtime deployment layers and execution dependencies.

3 CSS3 Wildcard Attribute Selector Mechanics

CSS3 attribute selectors evaluate element attributes using regular-expression-like substring operators directly inside the browser’s native C++ layout engine. This execution path is signif-

icantly faster than parsing complex XPath expressions and offers greater resilience against layout restructuring.

3.1 Prefix Matching Operator: `[attribute^='value']`

The prefix operator matches any element whose designated attribute value begins strictly with the specified substring prefix.

- **Syntax Formulation:** `[id^='checkBoxOption']`
- **Target Mapping:** Efficiently captures `checkBoxOption1`, `checkBoxOption2`, and `checkBoxOption3` under a single query without iterating individual IDs.
- **Practical Use Case:** Capturing UI form clusters, dynamic tabs, or framework-generated inputs that share common namespaces across deployments.

3.2 Substring Contains Operator: `[attribute*='value']`

The contains operator matches elements where the target substring appears anywhere within the attribute value, whether at the beginning, middle, or end.

- **Syntax Formulation:** `[id*='BoxOption']`
- **Target Mapping:** Captures all three checkbox inputs regardless of leading or trailing characters.
- **Practical Use Case:** Navigating component IDs embedded between timestamps or random GUIDs (e.g., `user_a8f9c_submit`).

3.3 Suffix Matching Operator: `[attribute$='value']`

The suffix operator evaluates the trailing characters of an attribute, matching when the attribute ends with the provided substring.

- **Syntax Formulation:** `[id$='Option3']`
- **Target Mapping:** Uniquely singles out the third checkbox input (`checkBoxOption3`) without specifying the prefix.
- **Practical Use Case:** Pinpointing specific elements whose index or state flag appears at the end of dynamic identifiers.

4 Locator Verification Matrix

The test harness executed four distinct wildcard queries, combining multiple element retrievals with singleton element lookups and subsequent user actions.

5 Runtime Execution Log & Telemetry Output

The automation script was executed within the virtual environment on Arch Linux. Below is the unedited terminal stream showing instantaneous stdout delivery, wildcard matches, and element state verification.

```
.venv > python Assignment_3.py
[BOOT] Launching CSS Selector Wildcard Challenge script...
=====
```

Strategy	CSS Selector	Matched	Target Tag	Verified Property	Action /
Prefix (^=)	[id^='checkBoxOption']	3 elements	<input>	Clicked 3; confirmed checked=True	all
Contains (*=)	[id*='BoxOption']	3 elements	<input>	Validated ID string contents	
Suffix (\$=)	[id\$='Option3']	1 element	<input>	Verified targeted ID match (Option3)	
Prefix (^=)	[name^='radioButton']	3 elements	<input>	Clicked index 0; checked radio1	

Table 2: CSS wildcard selector evaluation matrix and state results.

Target Component	Evaluated ID Attribute	Initial State	Post-Click State	Integrity Check
Checkbox #1	checkBoxOption1	False	True	Passed
Checkbox #2	checkBoxOption2	False	True	Passed
Checkbox #3	checkBoxOption3	False	True	Passed
Radio Button #1	radio1	False	True	Passed

Table 3: Component state-transition verification during automated interaction.

```

STARTING CSS WILDCARD ATTRIBUTE SELECTOR DEMO
=====
[*] Navigating to: https://rahulshettyacademy.com/AutomationPractice/
[\checkmark] Page loaded successfully.

-----
1. STARTS-WITH SELECTOR: [id^='checkBoxOption']
-----
[\checkmark] Found 3 checkboxes matching prefix [id^='checkBoxOption']
  -> Checkbox #1 (id='checkBoxOption1') clicked | checked=True
  -> Checkbox #2 (id='checkBoxOption2') clicked | checked=True
  -> Checkbox #3 (id='checkBoxOption3') clicked | checked=True

-----
2. CONTAINS SELECTOR: [id*='BoxOption']
-----
[\checkmark] Found 3 elements containing substring 'BoxOption':
  -> Tag: <input> | id: 'checkBoxOption1'
  -> Tag: <input> | id: 'checkBoxOption2'
  -> Tag: <input> | id: 'checkBoxOption3'

-----
3. ENDS-WITH SELECTOR: [id$='Option3']
-----
[\checkmark] Successfully matched ends-with element: id='checkBoxOption3'

-----
4. RADIO BUTTON SELECTOR: [name^='radioButton']
-----
[\checkmark] Found 3 radio buttons via [name^='radioButton']
  -> Clicked first radio button (value='radio1')
  -> Selected status: True

[SUCCESS] All wildcard selector operations completed successfully!

[*] Closing Firefox session...
[\checkmark] Session terminated cleanly.

```

6 Engineering Takeaways & Best Practices

The experiment establishes concrete architectural guidelines for authoring resilient locator strategies in modern automated testing pipelines:

- **Mitigating Dynamic Flakiness:** Hardcoded static locators break when UI frameworks append non-deterministic build hashes to element attributes. Employing prefix (^=) and contains (*=) operators bridges the gap between clean test automation and dynamic client-side rendering.
- **Performance Superiority over XPath:** CSS selectors execute via native browser engine query APIs (`document.querySelector()`), avoiding the XML evaluation overhead required by complex XPath engines.
- **Guarding Against Ambiguity:** Overly broad wildcard strings (such as matching a single character or common word) risk matching non-target elements. Best practice pairs the wildcard attribute with a tag name or container scope (e.g., `fieldset[id^='checkboxOption']`).
- **Terminal Buffering in Automated Pipelines:** Enabling `flush=True` on standard output streams is critical in POSIX CI/CD environments to guarantee real-time logging and avoid false timeout detections during long-running browser operations.